P3103R1
More bitset operations

Published Proposal, 2024-03-07

This version:
https://eisenwave.github.io/cpp-proposals/more-bitset-operations.html

Author:
Jan Schultke

Audience:
SG18, LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:
eisenwave/cpp-proposals

Abstract

This paper proposes adding a counterpart `std::bitset` member function for the utility functions in `<bit>`.

Table of Contents

§ 1. Revision history

§ 1.1. Changes since R0

- Simplified proposed wording by defining `rotl` as an equivalence in terms of `rotr`.
- Fixed minor editorial mistakes.

§ 2. Introduction

[P0553R4] added the bit manipulation library to C++20, which introduced many useful utility functions.

Some of these already have a counterpart in `std::bitset` (such as `popcount` as `bitset::count`), but not nearly all of them. This leaves `bitset` overall lacking in functionality, which is unfortunate because `std::bitset` is an undeniably useful container. At the time of writing, it is used in 73K files on GitHub; see [GitHub1].

`std::bitset` does not (and should not) expose the underlying integer sequence of the implementation. Therefore, it is not possible for the user to implement these operations efficiently themselves.

§ 3. Proposed changes

For each of the functions from the bit manipulation library that are not yet available in `std::bitset`, add a member function. Add a small amount of further useful member functions.

`<bit>` function	Proposed `bitset` member
`std::has_single_bit(T)`	`one()`
`std::countl_zero(T)`	`countl_zero()`
	`countl_zero(size_t)`
`std::countl_one(T)`	`countl_one()`
	`countl_one(size_t)`
`std::countr_zero(T)`	`countr_zero()`
	`countr_zero(size_t)`
`std::countr_one(T)`	`countr_one()`
	`countr_one(size_t)`
`std::rotl(T, int)`	`rotl(size_t)`
`std::rotr(T, int)`	`rotr(size_t)`
	`reverse()`

The additional overloads for the counting functions allow counting from a starting position. This can be useful for iterating over all set bits:

```
bitset<128> bits;
for (size_t i = 0; i != 128; ++i) {
    i += bits.countr_zero(i);
    if (i == 128) break;
    // ...
}
```

> NOTE: `byteswap` and `bit_cast` counterparts are not proposed, only functions solely dedicated to the manipulation of bit sequences.

§ 4. Design considerations

The naming of the member functions is based on the naming scheme in the `<bit>` header.

§ 4.1. `reverse`

`reverse` is a notable exception, which does not yet exist in `<bit>`. This function is added because the method for reversing integers may be tremendously faster than doing so bit by bit. ARM has a dedicated `RBIT` instruction for reversing bits, which could be leveraged.

I have written a not yet published proposal [Schultke1] which adds a corresponding bit-reversal function to `<bit>`.

§ 4.2. Counting overloads

`countl_zero`, `countr_zero`, `countl_one`, and `countr_one` are overloaded member functions which take a `size_t` argument or nothing.

This is preferable to a single member function with a defaulted argument because the overloads have different `noexcept` specifications. The overloads which take no arguments have a wide contract and can be marked `noexcept`, whereas the overloads taking `size_t` may throw `out_of_range`.

§ 5. Impact on existing code

The semantics of existing `bitset` member functions remain unchanged, and no existing valid code is broken.

This proposal is purely an extension of the functionality of `bitset`.

§ 6. Implementation experience

[Bontus1] provides a `std::bitset` implementation which supports most proposed features. There are no obvious obstacles to implementing the new features in common standard library implementations.

§ 7. Proposed wording

The proposed changes are relative to the working draft of the standard as of [N4917].

Modify the header synopsis in subclause 17.3.2 [version.syn] as follows:

```
#define __cpp_lib_bitset                              ~~202306L~~20XXXXL // als
```

Modify the header synopsis in subclause 22.9.2.1 [template.bitset.general] as follows:

```
namespace std {
  template<size_t N> class bitset {
  public:
    // bit reference
    [...]

    // [bitset.members], bitset operations
    constexpr bitset& operator&=(const bitset& rhs) noexcept;
    constexpr bitset& operator|=(const bitset& rhs) noexcept;
    constexpr bitset& operator^=(const bitset& rhs) noexcept;
    constexpr bitset& operator<<=(size_t pos) noexcept;
    constexpr bitset& operator>>=(size_t pos) noexcept;
    constexpr bitset  operator<<(size_t pos) const noexcept;
    constexpr bitset  operator>>(size_t pos) const noexcept;
+   constexpr bitset& rotl(size_t pos) noexcept;
+   constexpr bitset& rotr(size_t pos) noexcept;
+   constexpr bitset& reverse() noexcept;
    constexpr bitset& set() noexcept;
    constexpr bitset& set(size_t pos, bool val = true);
    constexpr bitset& reset() noexcept;
    constexpr bitset& reset(size_t pos);
    constexpr bitset  operator~() const noexcept;
    constexpr bitset& flip() noexcept;
    constexpr bitset& flip(size_t pos);

    // element access
    [...]

    // observers
    constexpr size_t count() const noexcept;
+   constexpr size_t countl_zero() const noexcept;
+   constexpr size_t countl_zero(size_t pos) const;
+   constexpr size_t countl_one() const noexcept;
+   constexpr size_t countl_one(size_t pos) const;
+   constexpr size_t countr_zero() const noexcept;
+   constexpr size_t countr_zero(size_t pos) const;
+   constexpr size_t countr_one() const noexcept;
+   constexpr size_t countr_one(size_t pos) const;
    constexpr size_t size() const noexcept;
    constexpr bool operator==(const bitset& rhs) const noexcept;
    constexpr bool test(size_t pos) const;
```

```
    constexpr bool all() const noexcept;
    constexpr bool any() const noexcept;
    constexpr bool none() const noexcept;
+   constexpr bool one() const noexcept;
  };

  // [bitset.hash], hash support
  template<class T> struct hash;
  template<size_t N> struct hash<bitset<N>>;
}
```

Modify subclause 22.9.2.3 [bitset.members] as follows:

> `constexpr bitset operator>>(size_t pos) const noexcept;`
>
> *Returns*: `bitset(*this) >>= pos`.
>
> `constexpr bitset& rotl(size_t pos) noexcept;`
>
> *Effects*: Equivalent to `rotr(N - pos % N)`.
>
> `constexpr bitset& rotr(size_t pos) noexcept;`
>
> *Effects*: Replaces each bit at position `I` in `*this` with the bit at position `(static_cast<U>(pos) + static_cast<U>(I)) % N`, where `U` is a hypothetical unsigned integer type whose width is greater than the width of `size_t`.
>
> *Returns*: `*this`.
>
> `constexpr bitset& reverse() noexcept;`
>
> *Effects*: Replaces each bit at position `I` in `*this` with the bit at position `N - I - 1`.
>
> *Returns*: `*this`.
>
> [...]
>
> `constexpr size_t count() noexcept;`
>
> *Returns*: A count of the number of bits set in `*this`.
>
> `constexpr size_t countl_zero(size_t pos) const;`
>
> *Returns*: The number of consecutive zero-bits in `*this`, starting at position `pos`, and traversing `*this` in decreasing position direction.
>
> *Throws*: `out_of_range` if `pos` does not correspond to a valid bit position.
>
> `constexpr size_t countl_zero() const noexcept;`
>
> *Returns*: `countl_zero(N - 1)`.

`constexpr size_t countl_one(size_t pos) const;`

Returns: The number of consecutive one-bits in `*this`, starting at position `pos`, and traversing `*this` in decreasing position direction.

Throws: `out_of_range` if `pos` does not correspond to a valid bit position.

`constexpr size_t countl_one() const noexcept;`

Returns: `countl_one(N - 1)`.

`constexpr size_t countr_zero(size_t pos) const;`

Returns: The number of consecutive zero-bits in `*this`, starting at position `pos`, and traversing `*this` in increasing position direction.

Throws: `out_of_range` if `pos` does not correspond to a valid bit position.

`constexpr size_t countr_zero() const noexcept;`

Returns: `countr_zero(0)`.

`constexpr size_t countr_one(size_t pos) const;`

Returns: The number of consecutive one-bits in `*this`, starting at position `pos`, and traversing `*this` in increasing position direction.

Throws: `out_of_range` if `pos` does not correspond to a valid bit position.

`constexpr size_t countr_one() const noexcept;`

Returns: `countr_one(0)`.

[...]

`constexpr bool none() const noexcept;`

Returns: `count() == 0`.

`constexpr bool one() const noexcept;`

Returns: `count() == 1`.

NOTE: The use of a hypothetical integer type in the specification of `rotl` and `rotr` is necessary because `(I + pos) % N` would be incorrect when `I + pos` wraps.

§ References

§ Normative References

[N4917]
Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 5 September 2022. URL: https://wg21.link/n4917

§ Informative References

[Bontus1]
Claas Bontus; d-xo; zencatalyst. *bitset2: bitset improved*. URL: https://github.com/ClaasBontus/bitset2

[GitHub1]
GitHub Code Search. *std::bitset language:c++*. URL: https://github.com/search?type=code&auto_enroll=true&q=%22std%3A%3Abitset%22+language%3Ac%2B%2B&p=1

[P0553R4]
Jens Maurer. *Bit operations*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0553r4.html

[Schultke1]
Jan Schultke. *Bit permutations*. URL: https://eisenwave.github.io/cpp-proposals/bit-permutations.html